

Authentication vs Authorization - Cheat Sheet

Authentication answers “who are you?” Authorization answers “what are you allowed to do?” Almost every security bug comes from mixing these up or skipping one.

The core distinction

Aspect	Authentication (AuthN)	Authorization (AuthZ)
Question	Who is making the request?	What can they do?
When	Usually once per session or token	Every protected operation
Failure	401 Unauthorized	403 Forbidden
Examples	Login, JWT verification, OAuth flow	Role check, permission check, ACL lookup
Compromise impact	Imposter access	Privilege escalation

Mnemonic: N for “name”, Z for “zones you can access”.

You always do authentication first. Authorization decisions depend on knowing who the caller is.

Authentication methods

Passwords

The classic. User submits a password. Server verifies against a stored hash.

Rules:

- Hash with a strong KDF: bcrypt, scrypt, or Argon2. Plain SHA-256 is fast enough to brute-force at scale.
- Salt every hash. Per-user salts prevent rainbow tables.
- Slow the hash deliberately. Bcrypt with cost factor 12+. Verification takes about 100ms. That’s the point.
- Rate limit login attempts. Prevents brute force.
- Use generic error messages. “Invalid credentials” is safer than “User not found” because it doesn’t leak which usernames exist.

Tokens

After login, the server issues a token. The client sends it on every subsequent request.

Two flavors:

- Opaque tokens. A random string. The server stores the mapping (token to user, expiry, scope) in a session store.
- JWT (JSON Web Tokens). Self-contained. The server signs claims (user ID, expiry, scopes), the client carries them, the server verifies the signature.

Trade-offs:

Aspect	Opaque token	JWT
Server storage	Required (lookup per request)	Not required
Revocation	Just delete the session	Hard (token is valid until expiry)
Size	Small	Larger (JSON + signature)
Verification	DB / cache lookup	Stateless signature check
Best for	Web sessions with a session store	Stateless APIs and microservices

API keys

Long-lived secret strings tied to a service account. Sent in a header or query param.

```
Authorization: Api-Key sk_live_XXXXX
```

Use for server-to-server auth. Rotate regularly. Scope them to specific permissions.

OAuth 2.0 / OIDC

Industry standard for delegated authorization. Users authenticate with their identity provider (Google, GitHub, Okta) and authorize your app to access specific scopes.

OAuth 2.0 itself is authorization. OpenID Connect (OIDC) is a layer on top that handles authentication. (See the OAuth 2.0 section below for flows.)

mTLS (Mutual TLS)

Both client and server present certificates. The TLS handshake authenticates both ends.

Common in service-to-service inside a service mesh (Istio, Linkerd) and for high-security APIs (banking, healthcare).

Magic links and passkeys

Modern passwordless alternatives.

- Magic link: server emails a unique URL. Clicking it logs the user in.

- Passkey / WebAuthn: asymmetric cryptography stored on the device. Replaces passwords entirely.

JWT anatomy

A JWT has three parts, dot-separated:

```
header.payload.signature
```

Each part is base64url-encoded.

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

`alg` is the signing algorithm.

Payload (claims)

```
{
  "sub": "user-42",
  "iss": "https://auth.example.com",
  "aud": "api.example.com",
  "iat": 1716000000,
  "exp": 1716003600,
  "scope": "read:profile write:profile",
  "tenant": "acme"
}
```

Standard claims:

- `sub` (subject): user ID.
- `iss` (issuer): who created the token.
- `aud` (audience): who the token is for.
- `iat` (issued at): timestamp.
- `exp` (expiration): timestamp.
- `nbf` (not before): timestamp.
- `jti` (JWT ID): unique identifier for revocation lists.

Signature

```
HMACSHA256(  
    base64url(header) + "." + base64url(payload),  
    secret  
)
```

Or with RSA/ECDSA (asymmetric), the issuer signs with a private key. Receivers verify with the public key.

Verifying a JWT (Java)

```
JwtParser parser = Jwts.parserBuilder()  
    .setSigningKey(publicKey)  
    .requireAudience("api.example.com")  
    .requireIssuer("https://auth.example.com")  
    .build();  
Jws<Claims> jws = parser.parseClaimsJws(tokenString);  
Claims claims = jws.getBody();  
String userId = claims.getSubject();
```

Always verify:

1. Signature. Otherwise anyone can forge tokens.
2. `exp` . Reject expired tokens.
3. `iss` . Reject tokens from unknown issuers.
4. `aud` . A token meant for service A shouldn't authorize service B.
5. `alg` . Pin to the expected algorithm. The `alg: "none"` and algorithm confusion attacks are real.

OAuth 2.0 flows

OAuth 2.0 defines several flows for different client types.

Authorization code (server-side web apps)

The standard. Browser-based, with a server backend.

1. User clicks "Sign in with Google".
2. Your app redirects to Google with `client_id`, `scope`, `redirect_uri`.
3. User logs in at Google, approves scopes.
4. Google redirects back to your app with a code.
5. Your server exchanges the code for an access token (and optionally a refresh token) at Google's token endpoint, using its `client_secret`.
6. Your server uses the access token to call Google APIs.

Authorization code + PKCE (mobile / SPA)

Same as above, but PKCE (Proof Key for Code Exchange) protects the flow when there's no secure place to keep a `client_secret`.

The client generates a code verifier, sends a hash with the initial request, then proves it has the original verifier at token exchange.

Client credentials (service-to-service)

No user. The service authenticates with `client_id` and `client_secret` to get a token.

```
POST /oauth/token
grant_type=client_credentials
client_id=...
client_secret=...
scope=...
```

Refresh token

Long-lived token that gets exchanged for fresh access tokens. Lets you keep users logged in without long-lived access tokens.

Implicit flow (deprecated)

Old SPA flow. Gets the token directly via URL fragment. Insecure. Use authorization code + PKCE instead.

Authorization models

Once you know who the caller is, how do you decide what they can do?

RBAC (Role-Based Access Control)

Users have roles. Roles have permissions.

```
User -> Role -> Permission
alice -> admin -> [users.read, users.write, billing.manage]
bob   -> viewer -> [users.read]
```

Simple, covers most cases. Roles tend to multiply over time (`admin`, `admin-readonly`, `tenant-admin`, etc.).

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(long id) {
    // ...
}
```

ABAC (Attribute-Based Access Control)

Decisions based on attributes of the user, resource, action, and environment.

```
allow if user.department == resource.department
allow if user.location == "US" and time.hour BETWEEN 9 AND 17
```

More flexible than RBAC. Harder to reason about and audit.

ACL (Access Control List)

Per-resource list of who can do what.

```
file:report.pdf
  alice: read, write
  bob: read
  group:auditors: read
```

Good for fine-grained file/document permissions. Doesn't scale well to millions of resources.

PBAC (Policy-Based Access Control)

Centralized policies in a dedicated engine. OPA (Open Policy Agent) and Cedar are the standard tools.

```
# OPA Rego policy
allow {
  input.user.role == "admin"
}
allow {
  input.action == "read"
  input.user.department == input.resource.department
}
```

Policy lives outside the app. Easier to audit and change centrally.

ReBAC (Relationship-Based Access Control)

Permissions based on relationships between users and resources. Google Zanzibar pioneered this. SpiceDB and OpenFGA are open implementations.

```
folder:engineering -> owner -> user:alice
folder:engineering -> viewer -> group:engineers
group:engineers    -> member -> user:bob
```

Bob inherits viewer access on `folder:engineering` through the group. Scales to billions of relationships.

Best for collaborative apps (Google Docs, Notion) where access depends on sharing graphs.

Sessions vs tokens

Two ways to maintain authentication state across requests.

Server-side sessions

The server stores session state. The client gets an opaque session ID (in a cookie).

```
Cookie: SESSIONID=abc123
```

Every request: server looks up `abc123`, finds the user info.

Pros: revocation is instant (delete the session). Small client payload. Server controls everything.

Cons: server must store state. Sharing across instances needs a session store (Redis).

Token-based (JWT)

The token is self-contained. Server verifies the signature, no lookup needed.

Pros: stateless, easy to scale horizontally. Standard for APIs and mobile.

Cons: revocation is hard. A leaked token is valid until it expires.

Hybrid is common: a short-lived access token (JWT) for API calls, a long-lived refresh token (opaque, stored server-side) for renewal.

Authorization in practice

The principles:

- Authenticate once, authorize everywhere. Every protected endpoint checks permissions.
- Default deny. No access unless explicitly granted.
- Least privilege. Users get exactly the permissions they need.
- Centralize the policy. Avoid scattering `if user.role == ...` everywhere.
- Audit logs. Record who did what, and when. Useful for compliance.

Common antipatterns:

- Trusting client-side checks. Hiding a button doesn't stop an API call. Always check server-side.
- Authorization in the controller. Security and business logic get tangled. Use middleware, decorators, or aspects.
- Per-tenant fields in every query. Easy to forget one. Use row-level security or tenant-aware repositories.
- Wrapping but not enforcing. A `@PreAuthorize` annotation that nobody validates is theater.

Best practices

- Always do AuthN before AuthZ. Authorization without knowing who is asking is undefined.
- Use established libraries. Spring Security, Auth0, Keycloak, Casbin. Roll-your-own auth is a security incident waiting.
- Hash passwords with bcrypt/scrypt/Argon2. Never store plain or fast-hashed passwords.
- Use short-lived access tokens (5-60 min) with refresh tokens. Limits the damage of a leak.
- Verify JWTs properly. Signature, expiry, issuer, audience, and the algorithm.
- Pin the signing algorithm. Don't trust the token's `alg` field. The library defaults can be too permissive.
- Rotate signing keys. Use a key ID (`kid`) in the JWT header. Publish public keys via JWKS.
- Default to deny. A new endpoint without explicit auth should fail closed.
- Audit access decisions. Log who accessed what, especially for sensitive data.
- Use MFA where it matters. Especially for admin accounts and high-risk operations.
- Rate limit auth endpoints. Brute force protection.

Common gotchas

- Confusing 401 with 403. 401 means "I don't know who you are". 403 means "I know who you are, you can't do this".
- JWT in localStorage. Vulnerable to XSS. Prefer httpOnly cookies for browser apps.
- Long-lived JWTs. A leaked 24-hour token is a 24-hour problem. Keep them short, use refresh tokens for renewal.
- Trusting the `alg` field. The `alg: "none"` attack and HS256/RS256 confusion attack come from libraries that trust the header.
- Not verifying `aud`. A token issued for service A gets used against service B because B doesn't check the audience.
- Role explosion. RBAC starts simple, grows into 50 roles nobody understands. Migrate to ABAC or ReBAC when complexity demands it.
- Authorization gaps in IDs. Endpoint accepts `GET /users/42` and forgets to check if the caller can see user 42. (Insecure Direct Object Reference, IDOR.)
- Mass assignment. Trusting client-supplied fields to set permissions or roles. Filter input.
- OAuth redirect URI not validated strictly. Open redirect equals OAuth token theft.
- Storing secrets in code. Client secrets, API keys, JWT signing keys. Use a secret manager.
- Mixing tenants in queries. A query that forgets `WHERE tenant_id = ?` returns data across tenants. Use row-level security.

Quick reference

Concept	Key fact
Authentication (AuthN)	Who are you?
Authorization (AuthZ)	What can you do?
401	Not authenticated
403	Authenticated but forbidden
Password storage	bcrypt / scrypt / Argon2 with per-user salt
Opaque token	Random string, server-side session lookup
JWT	Self-contained, signed claims
JWT verification	Signature + exp + iss + aud + alg
OAuth 2.0	Authorization framework
OIDC	Authentication layer on top of OAuth
Authorization code + PKCE	Standard SPA/mobile OAuth flow
Client credentials	Service-to-service OAuth flow
RBAC	Users have roles, roles have permissions
ABAC	Decisions based on attributes
ReBAC	Relationship-based (Zanzibar, OpenFGA)
MFA	Required for sensitive operations
Default deny	New endpoints fail closed